

Knowledge Representation

Lecture 5:

Reasoning about Actions V

dr Anna Maria Radzikowska

Wydział Matematyki i Nauk Informacyjnych

Pokój 504, gmach MiNI

E-mail: A.Radzikowska@mini.pw.edu.pl

Concurrent Actions

• *Conflict.*

What does it mean *conflicting actions*? In the theory of concurrency it is assumed that two processes accessing the same source are conflicting. In general case, however, this approach seems too restrictive. For example, two robots can pick up a heavy block simultaneously. What's more, this action may be unexecutable by each robot separately.

Problems (cont.)

- *Executability of actions.*

The conflict problem is a particular case of the one of actions' executability – clearly, conflicting actions cannot be executed concurrently. However, actions cannot be executed due to other reasons. For example, if there is a distance between the window and the door, I am not able to open them simultaneously.

Problems: effects of concurrent actions

Example 4.2. Mary is lifting a bowl of soup from the kitchen table, while John is opening the door to the dinning room.

These two actions are independent, so they can be performed simultaneously leading to the effect being the conjunction of effects of particular actions (i.e. the door is open and the bowl is lifted) – this way the *compound* action inherits results from its subactions.

Problems: effects of concurrent actions

Example 4.2. Mary is lifting a bowl of soup from the kitchen table, while John is opening the door to the dinning room.

These two actions are independent, so they can be performed simultaneously leading to the effect being the conjunction of effects of particular actions (i.e. the door is open and the bowl is lifted) – this way the *compound* action inherits results from its subactions.

Example 4.3. Whenever Mary tries to lift the bowl with one hand, she spills the soup (but lifted the bowl). But when she uses both hands, she does not spill the soup. We know that the soup is not spilled initially and the bowl is not lifted.

Now the effect “*the soup is spilled*” is not inherited when two actions, LIFTLEFT and LIFTRIGHT, are executed simultaneously.

Effects of concurrent actions (cont.)

Let D be an action domain and let the compound action $A = \{A_1, \dots, A_k\}$, $k \geq 2$, be given. Then

- **(inheritance)** if there is no statement in D describing effects of any subset of A , the concurrently executed actions lead to the effect being the conjunction of effects of particular actions.

Effects of concurrent actions (cont.)

Let D be an action domain and let the compound action $A = \{A_1, \dots, A_k\}$, $k \geq 2$, be given. Then

- **(inheritance)** if there is no statement in D describing effects of any subset of A , the concurrently executed actions lead to the effect being the conjunction of effects of particular actions.
- **(restricted inheritance)** If there is an action description statement in D referring to some subset $A' \subseteq A$, then the inheritance rule is not applied for elements of A' : A' leads to the effect given in statements in D referring to A' and no effect of the actions A' is composed of is taken into account. In other words, if such a statement is mentioned in D , it constitutes a “new action” (wrt its results).

The language $\mathcal{AC}$ – Syntax

- **Signature:** $\Upsilon = (\mathcal{F}, \mathcal{Ac})$, where $\mathcal{F}$ is the set of *fluent symbols* and $\mathcal{Ac}$ is the set of *action symbols*.

The language $\mathcal{AC}$ – Syntax

- **Signature**: $\Upsilon = (\mathcal{F}, \mathcal{Ac})$, where $\mathcal{F}$ is the set of *fluent symbols* and $\mathcal{Ac}$ is the set of *action symbols*.
- Each element $A \in \mathcal{Ac}$ is called an **atomic action** (*atom*, for short).

The language $\mathcal{AC}$ – Syntax

- **Signature:** $\Upsilon = (\mathcal{F}, \mathcal{A}_c)$, where $\mathcal{F}$ is the set of *fluent symbols* and $\mathcal{A}_c$ is the set of *action symbols*.
- Each element $A \in \mathcal{A}_c$ is called an **atomic action** (**atom**, for short).
- By a **compound action** (**action**) we mean any set $A \subseteq \mathcal{A}_c$ of actions.

The language $\mathcal{AC}$ – Syntax

- **Signature**: $\Upsilon = (\mathcal{F}, \mathcal{A}_c)$, where $\mathcal{F}$ is the set of *fluent symbols* and $\mathcal{A}_c$ is the set of *action symbols*.
- Each element $A \in \mathcal{A}_c$ is called an **atomic action** (*atom*, for short).
- By a **compound action** (*action*) we mean any set $A \subseteq \mathcal{A}_c$ of actions.
- Any statement of $\mathcal{AR}$, where instead of (atomic) actions we use compound actions, is a statement of $\mathcal{AC}$.

The language $\mathcal{AC}$ – Syntax

- **Signature**: $\Upsilon = (\mathcal{F}, \mathcal{A}_c)$, where $\mathcal{F}$ is the set of *fluent symbols* and $\mathcal{A}_c$ is the set of *action symbols*.
- Each element $A \in \mathcal{A}_c$ is called an **atomic action** (**atom**, for short).
- By a **compound action** (**action**) we mean any set $A \subseteq \mathcal{A}_c$ of actions.
- Any statement of $\mathcal{AR}$, where instead of (atomic) actions we use compound actions, is a statement of $\mathcal{AC}$.
- As before, a non-empty set of statements is called an **action domain**.

Elementary actions

Let D be an action domain. By an *elementary action in D* we mean any action $A \subseteq \mathcal{A}_c$ such that D contains any of the following statements:

A *causes* α *if* π

A *releases* f *if* π .

The set of all elementary actions in D will be denoted by *Elem*(D).

**What does it mean
“conflicting actions”?**

Example 4.4

Consider the following two atomic actions:

A *causes* $p \vee q$

B *causes* p .

Let $\sigma = \{\neg p, \neg q\}$. Performing of A in σ leads to two states:

$$\sigma_1 = \{p, \neg q\} \quad \sigma_2 = \{\neg p, q\},$$

(due to minimization of changes the state $\{p, q\}$ is not the resulting one), while performing B in σ leads to σ_1 .

Example 4.4

Consider the following two atomic actions:

A *causes* $p \vee q$

B *causes* p .

Let $\sigma = \{\neg p, \neg q\}$. Performing of A in σ leads to two states:

$$\sigma_1 = \{p, \neg q\} \quad \sigma_2 = \{\neg p, q\},$$

(due to minimization of changes the state $\{p, q\}$ is not the resulting one), while performing B in σ leads to σ_1 .

If we admit parallel execution of A and B in σ , then the resulting state will be σ_1 . Hence, while composing a nondeterministic action (A) with a deterministic action (B), we obtain a deterministic action – this is **counterintuitive!**

Example 4.5

Consider now the following two actions:

A causes $p \vee q$

B releases p .

Again, let $\sigma = \{\neg p, \neg q\}$. As before, performing A in σ leads to either of these states

$$\sigma_1 = \{p, \neg q\}, \quad \sigma_2 = \{\neg p, q\},$$

while executing B in σ leads to either to σ_1 or σ .

Example 4.5

Consider now the following two actions:

A causes $p \vee q$

B releases p .

Again, let $\sigma = \{\neg p, \neg q\}$. As before, performing A in σ leads to either of these states

$$\sigma_1 = \{p, \neg q\}, \quad \sigma_2 = \{\neg p, q\},$$

while executing B in σ leads to either to σ_1 or σ .

If we admit performing $\{A, B\}$, the resulting state will be σ_1 .

Then the composition of two nondeterministic actions makes a deterministic actions.

Problematic cases

Then the problem arises in the following two cases:

- D contains the following action description statements:

A **causes** α

B **releases** f

where

- $\alpha \Leftrightarrow \gamma_1 \vee \dots \vee \gamma_k$
- each $\gamma_i = \lambda_i^1 \wedge \dots \wedge \lambda_i^{k_i}$, where λ_i^j is a literal
- there exist $i = 1, \dots, k$ and $j = 1, \dots, k_i$ such that $f = \overline{\lambda_i^j}$.

In other words, f occurs in at least one of the possible effects of A .

Problematic cases (cont.)

- D contains the following two statements:

A **causes** α **if** π_1

B **causes** β **if** π_2

where

- $\alpha \Leftrightarrow \gamma_1 \vee \dots \vee \gamma_k$ and $\beta \Leftrightarrow \delta_1 \vee \dots \vee \delta_m$
- each $\gamma_i = \lambda_i^1 \wedge \dots \wedge \lambda_i^{k_i}$ and each $\delta_p = \tau_p^1 \wedge \dots \wedge \tau_p^{k_p}$, where λ_i^j and τ_p^l are literals
- there exist $i = 1, \dots, k$ and $p = 1, \dots, m$ such that γ_i and δ_p contain complementary literals (i.e. f and $\neg f$).

In other words, there is a possible effect of A (γ_i) **contradictory** to some possible effects of B (δ_p).

Disjunctive Normal Form (DNF)

Let α be any formula over $\mathcal{F}$. We say that α *is in DNF* iff

$$\alpha = \bigvee_{i=1}^n \bigwedge_{j=1}^{k(i)} \lambda_{ij},$$

where λ_{ij} is a literal.

E.g., $\alpha_1 = (p \wedge q) \vee \neg r$ is in DNF.

Disjunctive Normal Form (DNF)

Let α be any formula over $\mathcal{F}$. We say that α *is in DNF* iff

$$\alpha = \bigvee_{i=1}^n \bigwedge_{j=1}^{k(i)} \lambda_{ij},$$

where λ_{ij} is a literal.

E.g., $\alpha_1 = (p \wedge q) \vee \neg r$ is in DNF.

Theorem 4.1. Every formula α can be uniquely transformed to the formula α' in DNF.

Notation:

The formula resulting from transformation α into DNF will be denoted by $DNF(\alpha)$.

Disjunctive Normal Form (DNF)

Let α be any formula over $\mathcal{F}$. We say that α *is in DNF* iff

$$\alpha = \bigvee_{i=1}^n \bigwedge_{j=1}^{k(i)} \lambda_{ij},$$

where λ_{ij} is a literal.

E.g., $\alpha_1 = (p \wedge q) \vee \neg r$ is in DNF.

Theorem 4.1. Every formula α can be uniquely transformed to the formula α' in DNF.

Notation:

The formula resulting from transformation α into DNF will be denoted by $DNF(\alpha)$.

For $DNF(\alpha) = \gamma_1 \vee \dots \vee \gamma_k$, each γ_i is called a *component of α* .

Converting into DNF

1. Eliminate $\rightarrow$ and $\equiv$:

$$\alpha \rightarrow \beta \quad \rightsquigarrow \quad \neg\alpha \vee \beta$$

$$\alpha \equiv \beta \quad \rightsquigarrow \quad (\neg\alpha \vee \beta) \wedge (\neg\beta \vee \alpha).$$

2. Move $\neg$ to the right:

$$\neg(\alpha \wedge \beta) \quad \rightsquigarrow \quad \neg\alpha \vee \neg\beta$$

$$\neg(\alpha \vee \beta) \quad \rightsquigarrow \quad \neg\alpha \wedge \neg\beta$$

$$\neg\neg\alpha \quad \rightsquigarrow \quad \alpha.$$

3. Distribution $\vee$ wrt $\wedge$:

$$\alpha \wedge (\beta \vee \gamma) \quad \rightsquigarrow \quad (\alpha \wedge \beta) \vee (\alpha \wedge \gamma)$$

$$(\beta \vee \gamma) \wedge \alpha \quad \rightsquigarrow \quad (\beta \wedge \alpha) \vee (\gamma \wedge \alpha).$$

Conflicting actions

Let D be an action domain, let σ be a state, and let $A, B \subseteq \mathcal{A}_c$ be two actions. We say that A and B are **conflicting in σ** iff at least one of the following conditions holds:

- there are two statements in D of the form:

A **causes** α **if** π_1 ;

B **causes** β **if** π_2

such that

- $\sigma \models \pi_1 \wedge \pi_2$ and
- some component of α is inconsistent with some component of β .

Conflicting actions (cont.)

- there are statements in D of the form:

A causes α if π_1 ;

B releases f if π_2

such that

- $\sigma \models \pi_1 \wedge \pi_2$
- f occurs in some component of α .

Example 4.6

Lack of conflict does not mean that actions are executable in a state. Consider the following statements:

PAINTDOOR *causes* pd ;

PAINTWALL *causes* pw ;

PAINTWINDOW *causes* pwd ;

impossible {PAINTDOOR, PAINTWINDOW}.

Let $\sigma = \{\neg pd, \neg pw, \neg pwd\}$. Is it possible to execute $A = \{\text{PAINTWALL}, \text{PAINTWINDOW}, \text{PAINTDOOR}\}$ in σ ?

Example 4.6 (cont.)

There are 2 possible answers:

1. neither A nor any of its supersets is executable
2. A becomes a non-deterministic action: either $\{\text{PAINTDOOR}, \text{PAINTWALL}\}$ or $\{\text{PAINTWINDOW}, \text{PAINTWALL}\}$ is actually performed.

In the following we will consider the second case.

Potentially executable actions

Let D be an action domain and let σ be a state. An action $\{A_1, \dots, A_k\}$, $A_i \in \text{Elem}(D)$, is **potentially executable in σ** iff there is no $B \subseteq A_1 \cup \dots \cup A_k$ such that D contains statements:

B *causes* α_1 *if* π_1

...

B *causes* α_n *if* π_n

such that

- $\sigma \models \pi_1 \wedge \dots \wedge \pi_k$
- $\alpha_1 \wedge \dots \wedge \alpha_k$ is inconsistent.

Example 4.7

Consider the following statements:

A causes p if q;

B causes $\neg q$ if q;

C causes s if q;

$\{A, B\} \neg r$ if q;

impossible $\{A, C\}$ if r.

Let $\sigma \models q \wedge r$. Elementary actions $\{A, B\}$ and $\{C\}$ are not potentially executable in σ , but $\{B\}$ and $\{C\}$ are potentially executable.

Decomposition of actions

Let D be an action domain, let σ be a state, and let $A \subseteq \mathcal{A}_c$. A **decomposition of A in σ** is a set $\Delta(A, \sigma) = \{E_1, \dots, E_k\}$ of elementary actions in D such that

(D.1) $\bigcup_{i=1}^k E_i \subseteq A$

(D.2) $\Delta(A, \sigma)$ is the set non-conflicting actions potentially executable in σ

(D.3) there is no set $\{E'_1, \dots, E'_k, E'_{k+1}, \dots, E'_m\}$ of actions satisfying the above two conditions and, in addition, $E_i \subset E'_i$ for some $i = 1, \dots, k$.

Decomposition of actions

Let D be an action domain, let σ be a state, and let $A \subseteq \mathcal{A}_c$. A **decomposition of A in σ** is a set $\Delta(A, \sigma) = \{E_1, \dots, E_k\}$ of elementary actions in D such that

(D.1) $\bigcup_{i=1}^k E_i \subseteq A$

(D.2) $\Delta(A, \sigma)$ is the set non-conflicting actions potentially executable in σ

(D.3) there is no set $\{E'_1, \dots, E'_k, E'_{k+1}, \dots, E'_m\}$ of actions satisfying the above two conditions and, in addition, $E_i \subset E'_i$ for some $i = 1, \dots, k$.

Note that **(D.3)** says about the maximality of $\Delta(A, \sigma)$.

Decomposition of actions

Let D be an action domain, let σ be a state, and let $A \subseteq \mathcal{A}_c$. A **decomposition of A in σ** is a set $\Delta(A, \sigma) = \{E_1, \dots, E_k\}$ of elementary actions in D such that

(D.1) $\bigcup_{i=1}^k E_i \subseteq A$

(D.2) $\Delta(A, \sigma)$ is the set non-conflicting actions potentially executable in σ

(D.3) there is no set $\{E'_1, \dots, E'_k, E'_{k+1}, \dots, E'_m\}$ of actions satisfying the above two conditions and, in addition, $E_i \subset E'_i$ for some $i = 1, \dots, k$.

Note that **(D.3)** says about the maximality of $\Delta(A, \sigma)$.

If $\Delta(A, \sigma) = \{E_1, \dots, E_k\}$ is the decomposition of A in σ , then each E_i , $i = 1, \dots, k$, is called a **component of executing of A in σ** .

Example 4.8

Consider the following action domain D :

A **causes** $\neg p$ **if** p ;
 B **causes** q **if** p ;
 C **causes** r **if** p ;
 D **causes** w **if** p ;
 $\{A, B\}$ **causes** v **if** p ;
 $\{B, C\}$ **causes** s **if** p ;
 $\{A, B, C\}$ **causes** f **if** p ;
impossible $\{A, C\}$.

Let $F = \{A, B, C, D\}$ and let $\sigma \models p$. There are two decompositions of F in σ , namely

$$\Delta_1(F, \sigma) = \{\{A, B\}, \{D\}\}$$

$$\Delta_2(F, \sigma) = \{\{B, C\}, \{D\}\}.$$

Potentially resulting states

Let D be an action domain, let σ, σ' be states, let $A \subseteq \mathcal{Ac}$ be an action, and let $\Delta(A, \sigma) = \{E_1, \dots, E_k\}$ be a decomposition of A in σ . We say that σ' is a **potentially resulting state of execution of A in σ** iff for every $i = 1, \dots, k$ and for every statement

$$(E_i \text{ causes } \alpha \text{ if } \pi) \in D,$$

$$\sigma \models \pi \text{ implies } \sigma' \models \alpha.$$

Intuitively, while performing A in σ , we get a (potentially) resulting state satisfying all effects of its components E_i in σ , which belong to the same decomposition.

Potentially resulting states (cont.)

In other words, if A has n distinct decompositions

$$\Delta_1(A, \sigma) = \{E_1^1, \dots, E_{k_1}^1\}$$

...

$$\Delta_n(A, \sigma) = \{E_1^n, \dots, E_{k_n}^n\}$$

and the execution of E_i^j in σ has the effect α_i^j , then the potentially resulting state σ' (of the performance of A in σ) satisfies

$$(\alpha_1^1 \wedge \dots \wedge \alpha_{k_1}^1) \vee \dots \vee (\alpha_1^n \wedge \dots \wedge \alpha_{k_n}^n).$$

Example 4.8 (cont.)

A causes $\neg p$ if p ;
B causes q if p ;
C causes r if p ;
D causes w if p ;
 $\{A, B\}$ causes v if p ;
 $\{B, C\}$ causes s if p ;
 $\{A, B, C\}$ causes f if p ;
impossible $\{A, B\}$.

Executing $F = \{A, B, C, D\}$ in σ potentially leads to two states σ' and σ'' such that

$$\begin{aligned}\sigma' &\models v \wedge w \\ \sigma'' &\models s \wedge w.\end{aligned}$$

Hence, after F , we for sure have w and either v or s .

Structures for $\mathcal{AC}$

A *structure for $\mathcal{AC}$* is a system (Σ, σ_0, Res) such that Σ is a set of states, $\sigma_0 \in \Sigma$ is an initial state, and $Res : 2^{\mathcal{A}^c} \times \Sigma \rightarrow 2^\Sigma$.

Structures for $\mathcal{AC}$

A **structure for $\mathcal{AC}$** is a system (Σ, σ_0, Res) such that Σ is a set of states, $\sigma_0 \in \Sigma$ is an initial state, and $Res : 2^{\mathcal{AC}} \times \Sigma \rightarrow 2^\Sigma$.

Let D be an action domain in $\mathcal{AC}$. As before, in order to define a model of D , we define an auxiliary mapping $Res_0 : 2^{\mathcal{AC}} \times \Sigma \rightarrow 2^\Sigma$ as follows: for every $\sigma \in \Sigma$ and for every $A \subseteq \mathcal{AC}$,

if A has in σ n decompositions $\Delta_1(A, \sigma), \dots, \Delta_n(A, \sigma)$,
 $\Delta_i(A, \sigma) = \{A_1^i, \dots, A_{k_i}^i\}$, then $Res_0(A, \sigma)$ is the set of all
potentially resulting states of the performance of A in σ .

Structures for $\mathcal{AC}$

A **structure for $\mathcal{AC}$** is a system (Σ, σ_0, Res) such that Σ is a set of states, $\sigma_0 \in \Sigma$ is an initial state, and $Res : 2^{\mathcal{Ac}} \times \Sigma \rightarrow 2^\Sigma$.

Let D be an action domain in $\mathcal{AC}$. As before, in order to define a model of D , we define an auxiliary mapping $Res_0 : 2^{\mathcal{Ac}} \times \Sigma \rightarrow 2^\Sigma$ as follows: for every $\sigma \in \Sigma$ and for every $A \subseteq \mathcal{Ac}$,

if A has in σ n decompositions $\Delta_1(A, \sigma), \dots, \Delta_n(A, \sigma)$,
 $\Delta_i(A, \sigma) = \{A_1^i, \dots, A_{k_i}^i\}$, then $Res_0(A, \sigma)$ is the set of all
potentially resulting states of the performance of A in σ .

For any action $A \subseteq \mathcal{Ac}$ and for any two states $\sigma, \sigma' \in \Sigma$, the mapping $New(A, \sigma, \sigma')$ are defined as in the semantics of $\mathcal{AR}$.

Model

Let D be an action domain in $\mathcal{AC}$. A structure $S = (\Sigma, \sigma_0, Res)$ is a **model of D** iff

- (M.1) Σ is the set of all states for D (i.e. the set of all states satisfying all constraints from D);
- (M.2) each value statement and each observation statement is true in S (analogously to $\mathcal{AR}$);
- (M.3) for every action $A \subseteq \mathcal{AC}$ and for every state $\sigma \in \Sigma$,
 $Res(A, \sigma) \subseteq Res_0(A, \sigma)$ is the set of all states σ' , for which the sets $New(A, \sigma, \sigma')$ are minimal (wrt set inclusion).

Example 4.9: Producers–Consumers

Assume that there is one producer and two consumers, *A* and *B*. The producer puts to the buffer as many items of his product as possible by performing an action `PUT`. Each consumer can perform two actions: `CONSUME` and `GET`. The action `GET` makes the buffer empty, while the action `CONSUME` causes that the consumer does not have any item.

Example 4.9: Producers–Consumers

Assume that there is one producer and two consumers, A and B . The producer puts to the buffer as many items of his product as possible by performing an action PUT . Each consumer can perform two actions: CONSUME and GET . The action GET makes the buffer empty, while the action CONSUME causes that the consumer does not have any item.

Representation in $\mathcal{AC}$:

PUT **causes** $\neg \text{empty}$;

GETA **causes** $\text{empty} \wedge \text{hasA}$ **if** $\neg \text{empty}$;

GETB **causes** $\text{empty} \wedge \text{hasB}$ **if** $\neg \text{empty}$;

CONSUMEA **causes** $\neg \text{hasA}$ **if** hasA ;

CONSUMEB **causes** $\neg \text{hasB}$ **if** hasB ;

impossible $\{\text{GETA}, \text{GETB}\}$.

Example 4.9 (cont.)

Consider a state $\sigma = \{hasA, hasB, \neg empty\}$ and the (compound) action $A = \text{PUT}, \text{GETA}, \text{GETB}, \text{CONSUMEA}, \text{ConsumeB}\}$.

Example 4.9 (cont.)

Consider a state $\sigma = \{hasA, hasB, \neg empty\}$ and the (compound) action $A = \text{PUT}, \text{GETA}, \text{GETB}, \text{CONSUMEA}, \text{ConsumeB}\}$.

The actions PUT and GETA , as well as the actions PUT and GETB , are conflicting in σ , so executing PUT makes GETA and GETB unexecutable.

Example 4.9 (cont.)

Consider a state $\sigma = \{hasA, hasB, \neg empty\}$ and the (compound) action $A = \text{PUT}, \text{GETA}, \text{GETB}, \text{CONSUMEA}, \text{ConsumeB}\}$.

The actions PUT and GETA , as well as the actions PUT and GETB , are conflicting in σ , so executing PUT makes GETA and GETB unexecutable.

Also, GETA and GETB are together unexecutable and either of these actions can be performed at a time.

Example 4.9 (cont.)

Consider a state $\sigma = \{hasA, hasB, \neg empty\}$ and the (compound) action $A = \text{PUT}, \text{GETA}, \text{GETB}, \text{CONSUMEA}, \text{ConsumeB}\}$.

The actions PUT and GETA , as well as the actions PUT and GETB , are conflicting in σ , so executing PUT makes GETA and GETB unexecutable.

Also, GETA and GETB are together unexecutable and either of these actions can be performed at a time.

Moreover, GETA and CONSUMEA , as well as GETB and ConsumeB , are conflicting.

Example 4.9 (cont.)

Hence, we obtain the following three decompositions of A in σ :

$$\Delta_1(A, \sigma) = \{\text{PUT}, \text{CONSUMEA}, \text{CONSUMEB}\}$$

$$\Delta_2(A, \sigma) = \{\text{GETA}, \text{CONSUMEB}\}$$

$$\Delta_3(A, \sigma) = \{\text{GETB}, \text{CONSUMEA}\}.$$

Hence $\text{Res}(A, \sigma) = \{\sigma_1, \sigma_2, \sigma_3\}$, where

$$\sigma_1 = \{\neg hasA, \neg hasB, \neg empty\}$$

$$\sigma_2 = \{hasA, \neg hasB, empty\}$$

$$\sigma_3 = \{\neg hasA, hasB, empty\}.$$

Thank you for your attention!

Any questions are welcome.